1. Problem statement

Motivation. The application areas for robotics continues to expand, even in household contexts. Users may not always be able to receive training on how to operate robotic systems, as formal training can be very time consuming for individuals and for companies. Intuitive forms of teleoperation aim to solve this, by reading an operator's movements/gestures and mapping these to corresponding commands (such as setpoints) for the robot systems. This introduces a new challenge; mapping human limb kinematics to corresponding kinematics for robotic limbs, which often aren't anthropomorphic. There are many mathematical methods (models and transformations) that solve this problem, but few generalised implementations [1][2]. Configuring and implementing these mathematical models remains a challenge and the implementation is often case-specific [3][4]. This project aims to develop a software framework that simplifies the configuration and implementation of one or many of these methods, thus reducing the time required to develop and re-configure teleoperation systems. In addition to developing the framework, this project will test it in a real-world use-case, to demonstrate its effectiveness. Since cybersecurity becomes a major concern in the context of teleoperated systems, this project briefly raises and discusses concerns about remote cyber-physical systems, using the real-world implementation as an example.

Context. This project aims to develop a software framework that simplifies the configuration and implementation of one or many of the mapping methods. This will allow more time and resources to be invested into cutting-edge technologies of teleoperation project, and make these kinda of projects more accessible to hobbyists who may not be proficient in kinematics. Optimistically, if the framework becomes widely adopted, it may introduce new standardisations to teleoperation, inviting experts to optimise the framework, and develop surrounding/cooperative frameworks/solutions.

Technical challenge. Some of the tehcnical challenges of this project are: (i) to develop the mapping framework to be easy/intuitive, (ii) versatile, (iii) and efficient. Challenges relating to the real-world testing include: (iv) implementing a computer vision system to detect the user's hand positions, (v) interfacing with the framework, (vi) wirelessly and securely communicating setpoints to a Braccio arm, (vii) and making the Braccio arm move to the given setpoints.

Limitations. Depending on the computer vision implementation, the user's workspace may be limited, such as needing a static plain background. The scope of this project does not include optimising the real-world demonstration.

2. Project requirements

2.1 Mission requirements of the product

The proposed system is a library of functions and data structures written in Rust. The Mission-critical requirements for this project are as follows:

- The program must successfully compile.
- The mapping algorithm must be valid.
  - The program must correctly map the position of one limb to a position for another limb, given a set rules specified by the user.
  - The program must work for an arbitrary pair of source and destination limbs.
  - The program's mapping must be smooth (continuous and differentiable) such that the destination limb does not jerk due to the algorithm.
- The mapping algorithm must work in realtime.
  - The program must create mappings from snapshots of positions, not entire trajectories.
  - The mapping algorithm must be fast; little contribution to latency.

2.2 Student tasks

The student is tasked with completing the following:

- Familiarise themself with various kinematic-mapping algorithms, and select / create one to be implemented.
- Then program a general implementation of the mapping algorithm, allowing for various kinematic models.
- Create a testbench to verify the correctness of the implementation, and test performance.
- Implement an arm-tracking computer vision algorithm, that gives a model of a human's hand.
- Wirelessly transmit setpoints from the student's laptop to the Braccio arm.
- Implement a simple control program that moves the Braccio arm to the received setpoints.
- Interface all of the subsystems.
- Perform an attack-surface analysis of the entire system.

3. Functional analysis

This section aims to give a detailed higher level overview of the overall real-world application of the proposed solution.

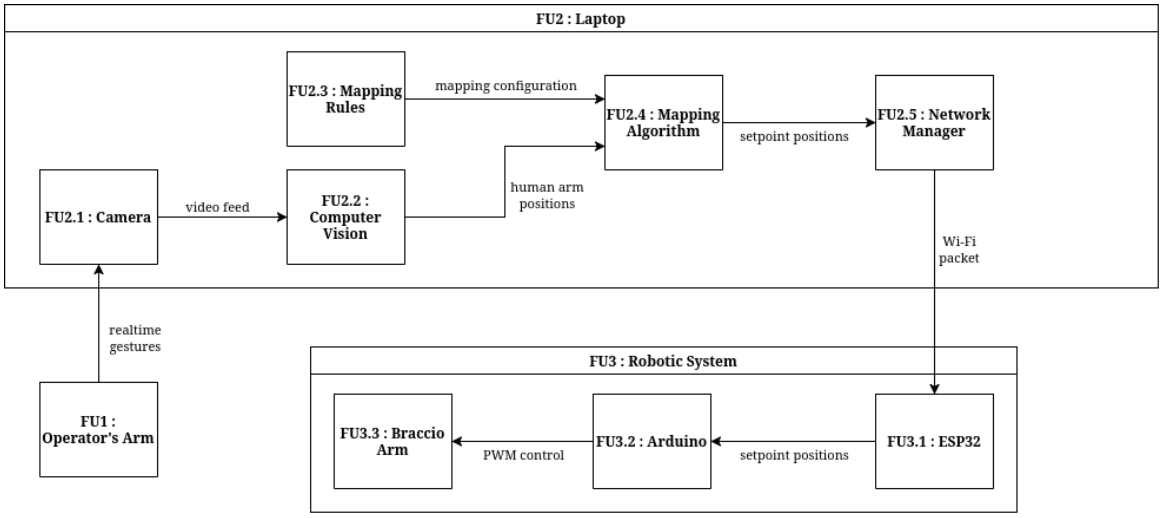


Figure 3.1: System Block Diagram

FU1, the operator's arm, serves as the primary input to the system. Its positions ultimately determine the setpoint for the Braccio arm (FU3.3).

FU2, the student's laptop, serves as the interface through which the operator's arm positions get captured, interpreted, and wirelessly sent to the robotic system (FU3). The primary components are: a camera, Wi-Fi antenna, and a software pipeline. ***FU2.1,*** the laptop's camera, captures the operator's arm's gestures/positions, and feeds this to the computer vision algorithm (FU2.2). ***FU2.2,*** the computer vision algorithm, processes the camera's video feed, and models the operator's arm's position. FU2.2 and FU2.1 will likely be handled by a singular open source solution. ***FU2.3,*** the mapping rules, is a file and/or data structure that specifies how the human arm's positions relate to the Braccio arm's positions, in a high-level of abstraction. This is the method that engineers would use to configure the mapping in other projects as well. ***FU2.4,*** the mapping algorithm, takes the operator's arm's positions, as well as the mapping configuration, to determine the best-approximation setpoints for the Braccio arm (FU3.3). ***FU2.5,*** the network manager, converts the setpoints into data packets and sends them to the ESP32 (FU3.1) over Wi-Fi.

FU3, the robotic system, is the remote actuator that is intended on being controlled, along with assisting peripherals. ***FU3.1,*** the ESP32, receives the Wi-Fi data packets, and gives them to the Arduino (FU3.2) over a serial communication protocol. The protocol will likely be UART, since there aren't multiple slave devices to communicate with, and full-duplex communication is not necessary. ***FU3.2,*** the Arduino, asynchronously receives setpoints from the ESP32 (FU3.1), while controlling the Braccio arm (FU3.3) with PWM signals (the Braccio uses servo motors). ***FU3.3***, the Braccio arm, is the actuator that's being controlled by the user. Its effective accuracy and speed is how the system's overall performance is measured.

4. Target specifications

4.1 Mission-critical system specifications

SPECIFICATIONS (in measureable terms)	**ORIGIN** (or motivation of this specification)	**VERIFICATION** (How will you confirm/measure that your system complies with this specification?)
The Braccio must track the human arm with no more than 0.2s of delay, excluding transmission latency over wireless protocol.	Teleoperation needs to be fast to feel responsive and intuitive. The 0.2s value is double the time given for the TeleopLab project [5]. Since teleoperation in practical contexts can happen over large distances, the transmission latency is unbounded.	Using software timestamps, the latency from video-image to transmision will be recorded. Again, using timestampes, the latency from reception to set-point satisfaction will be measured. This will be confirmed with a slowed down video recording.
The computation of mapping from human-arm space to Braccio-arm space mustn't take more than 0.1s.	The contribution of this project shouldn't be computationally expensive. The value 0.1s was the upper bound given for the inverse kinematics computation in the TeleopLab project [5].	Using software timestamps, the latency of the computation will be measured. The portion of contribution to latency will then be computed, using the total latency.
The movement's hysteresis must be less than 5%.	Movements/positions should be easily replicatable to be intuitive. A concern is that returning to the same position from a different angle/approach results in a different final-posiiton. The choice of 5% is somewhat arbitrary.	Move each of the joints to their maximums, then move them to the mid-point. Record the actual position as the upper hysteresis. Move each of the joints to their minimums, then move them to the mid-point. Record the actual position as the lower hysteresis. The difference between these recordings is double the hysteresis.

Table 4.1: Mission-critical system specifications

SPECIFICATIONS (in measurable terms)	**ORIGIN** (or motivation of this specification)	**VERIFICATION** (How will you confirm/measure that your system complies with this specification?)
The mapping program mustn't fail when a mapping does not exist or is not possible.	The robotic limb's kinematics may be very limited in some movements, compared to the human's limb. The program mustn't produce an unhandled exception from these scenarios.	Faulty human-to-robot mappings will be fed into the algorithm, and the algorithm mustn't disrupt the flow of the rest of the software pipeline. Examples of failure are: by crashing, or giving undefined/unpredictable behaviour.

Table 4.2: Mission-critical system specifications (continued)

4.2 Field conditions

REQUIREMENT	**SPECIFICATION** (in measureable terms)
There mustn't be distractions in the background that may make the computer vision struggle.	There needs to be a blank wall (painted brick, cardboard, et cetera) behind the user's arm.
The mount for the Braccio isn't very strong, and wind may loosen/break it. The wind mustn't topple the arm over.	The windspeed must be less than 20km/h.

Table 4.3: Field conditions

Bibliography

[1] C. Meeker, T. Rasmussen, and M. Ciocarlie, "Intuitive hand teleoperation by novice operators using a continuous teleoperation subspace," in *2018 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2018, pp. 5821–5827. [Online]. Available: https://ieeexplore.ieee.org/abstract/document/8460506

[2] C. Meeker, M. Haas-Heger, and M. Ciocarlie, "A continuous teleoperation subspace with empirical and algorithmic mapping algorithms for nonanthropomorphic hands," *IEEE Transactions on Automation Science and Engineering*, vol. 19, no. 1, pp. 373–386, 2020. [Online]. Available: https://ieeexplore.ieee.org/abstract/document/9258984

[3] M.-V. Drăgoi, A.-V. Frimu, A. Postelnicu, R.-A. Puiu, G. Petrea, and A. Hank, "Interactive teleoperation of an articulated robotic arm using vision-based human hand tracking," *Biomimetics*, vol. 11, no. 2, 2026. [Online]. Available: https://www.mdpi.com/2313-7673/11/2/151

[4] Z. Wang, I. Reed, and A. M. Fey, "Toward intuitive teleoperation in surgery: Human-centric evaluation of teleoperation algorithms for robotic needle steering," in *2018 IEEE International Conference on Robotics and Automation (ICRA)*, 2018, pp. 5799–5806. [Online]. Available: https://ieeexplore.ieee.org/abstract/document/8460729

[5] Z. Chen, Y. J. Yoon, R. Bautista-Montesano, Z. Zhao, A. Mandlekar, and J. Liu, "Teleoplab: Accessible and intuitive teleoperation of a robotic manipulator for remote labs," 2025. [Online]. Available: https://arxiv.org/abs/2509.05547